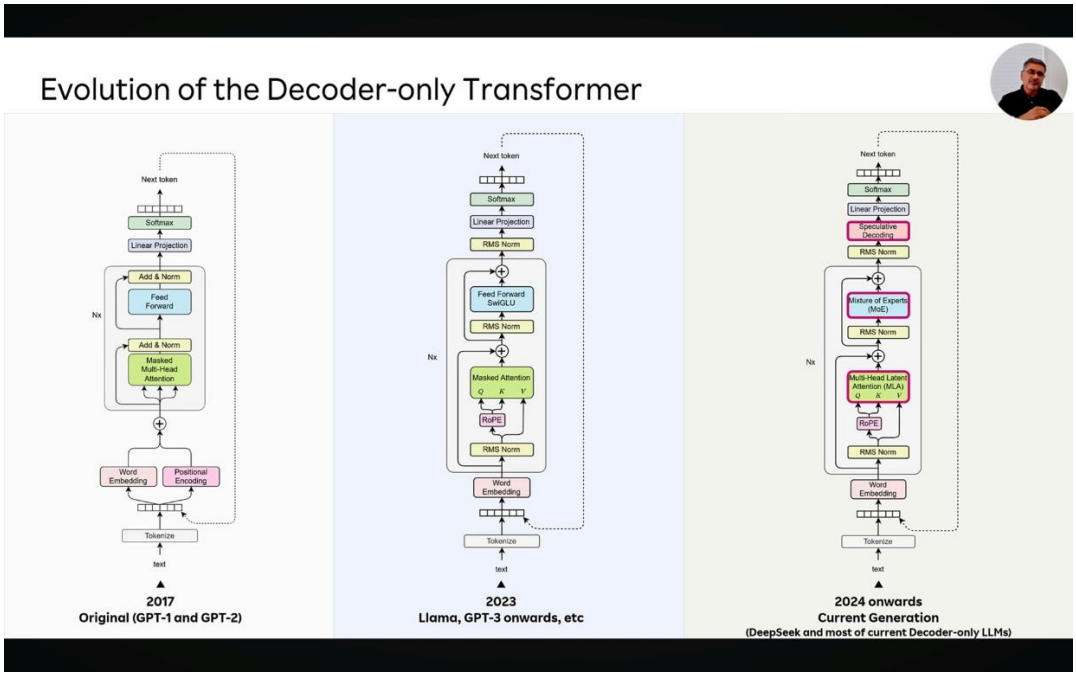
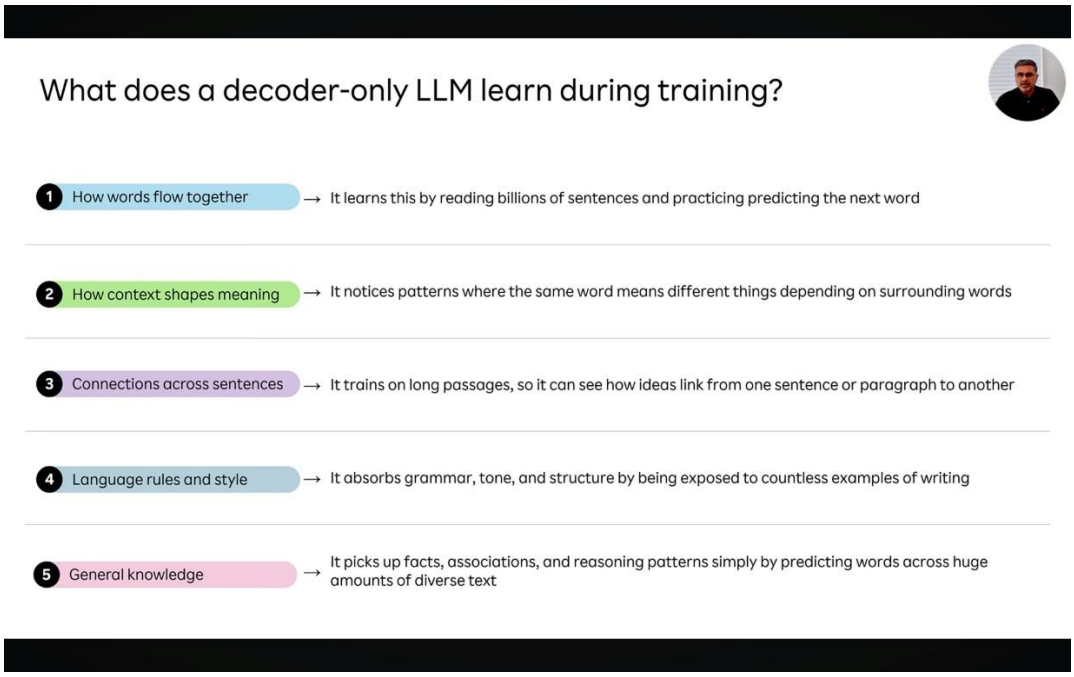
DECODER ONLY TRANSFORMER

A decoder-only Transformer is an autoregressive model trained to predict the next token using masked self-attention, producing a probability distribution over the vocabulary at each position, and is primarily used for scalable, flexible text generation.

It can also easily be defined as - A decoder-only Transformer is a neural network that reads tokens from left to right and predicts the next token, using masked self-attention so each token can see only past tokens.





PRE-TRAINING OF DECODER ONLY TRANSFORMER

(learn the pages form 3.3 [1,4,5])

TOKENIZATION

WORD EMBEDDINGS

RMS NORM

RMSNorm (Root Mean Square Normalization) is a normalization technique that scales activations based only on their root mean square, without subtracting the mean.

Mathematical Definition

Given a hidden vector $x \in \mathbb{R}^d$:

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}} \odot g$$

Where:

- d= hidden dimension
- ϵ= numerical stability constant
- $g \in \mathbb{R}^d$= learnable scale parameter

In LLaMA, the embedding vector is the initial hidden vector at layer 0, but hidden vectors at deeper layers are context-aware representations refined by causal self-attention and feed-forward networks.

ROPE (Rotary Positional Embeddings)

Why Position Information Is Needed

Self-attention is permutation-invariant by default.
Without position information, the sequence:

"I love NLP"
"NLP love I"

would look identical to attention. So positional information must be injected.

Definition

RoPE (Rotary Positional Embedding) encodes positional information by rotating query and key vectors in a position-dependent way before computing attention.

In RoPE, each token's query and key vectors are rotated by a position-dependent angle, so attention scores encode relative position without adding explicit positional embeddings.

Mathematical Intuition

Consider a hidden dimension split into pairs:

$$(x_1, x_2), (x_3, x_4), \ldots$$

Each pair is treated as a 2D vector.

Rotation Matrix

For position p and frequency θ:

$$R(p) = \begin{bmatrix} \cos(p\theta) & -\sin(p\theta) \\ \sin(p\theta) & \cos(p\theta) \end{bmatrix}$$

RoPE applies:

$$\tilde{q}_p = R(p)q$$
$$\tilde{k}_p = R(p)k$$

In RoPE, each token's query and key vectors are rotated by a position-dependent angle, so attention scores encode relative position without adding explicit positional embeddings.

CAUSAL MASKING

Causal masking prevents each token from attending to future tokens by masking the upper-triangular part of the attention matrix, enabling autoregressive training in LLaMA.

Why Causal Masking Is Critical in LLaMA

1. Prevents Information Leakage

- Model cannot access tokens it's supposed to predict

2. Enables Autoregressive Training

- Matches inference-time generation behaviour

3. Makes Parallel Training Possible

- All tokens processed at once
- Mask enforces causality

MASKED MULTI-HEAD ATTENTION WITH INCLUSION OF CASUAL MASKING AND ROPE

In LLaMA, masked multi-head self-attention projects hidden states into queries, keys, and values, applies rotary positional embeddings to queries and keys to encode relative positions, enforces causality via a triangular mask to block future tokens, and aggregates past information through weighted value sums, producing context-aware token representations suitable for autoregressive language modeling.

Working steps are –

Step 1: Linear Projections → Q, K, V

LLaMA projects the input hidden states into:

$$Q = XW_Q, K = XW_K, V = XW_V$$

Where:

- $W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$

What This Does

- Creates three different views of the same token:
  - Query: what this token is looking for
  - Key: what this token offers
  - Value: information to be passed forward

Step 2: Split into Multiple Heads

$$Q, K, V \rightarrow (Q_1, Q_2, \dots, Q_h)$$

Where:

- h = number of attention heads
- Each head operates on a subspace

Why Multi-Head?

- Different heads learn:
  - Syntax
  - Semantics
  - Long-range vs short-range dependencies

Step 3: Apply RoPE to Q and K (LLaMA-Specific)

What Happens

For each head:

$$Q_h \to \tilde{Q}_h \quad K_h \to \tilde{K}_h$$

Using rotary position-dependent rotations.

Why Only Q and K?

- Attention score depends on $QK^\top$
- RoPE makes this dot product encode relative position
- Values contain content, not position

Objective

Inject relative positional information directly into attention scores.

Step 4: Compute Scaled Dot-Product Attention Scores

$$S = \frac{\tilde{Q}_h \, \tilde{K}_h}{\sqrt{d_k}}$$

Why Scaling?

- Prevents large dot products
- Stabilizes softmax gradients

Step 5: Apply Causal Mask (Critical Step)

Mask Definition

$$M_{i,j} = \begin{cases} 0 & j \le i \\ -\infty & j > i \end{cases}$$

Masked Scores

$$S' = S + M$$

What This Does

- Blocks attention to future tokens
- Enforces autoregressive constraint

Step 6: Softmax → Attention Weights

$$A = \text{softmax}(S')$$

Properties:

- Row-wise
- Masked positions become zero probability

Step 7: Weighted Sum of Values

$$O = AV$$

What This Produces

- Each token receives a context-aware representation
- Built only from past tokens

Step 8: Concatenate Heads + Output Projection

$$O_{\text{concat}} = \text{Concat}(O_1, \dots, O_h)$$
$$O_{\text{final}} = O_{\text{concat}} W_O$$

This produces:

$$O_{\text{final}} \in \mathbb{R}^{T \times d_{\text{model}}}$$

Mandatory, visualize above step from – 3.3 [48 – 60]

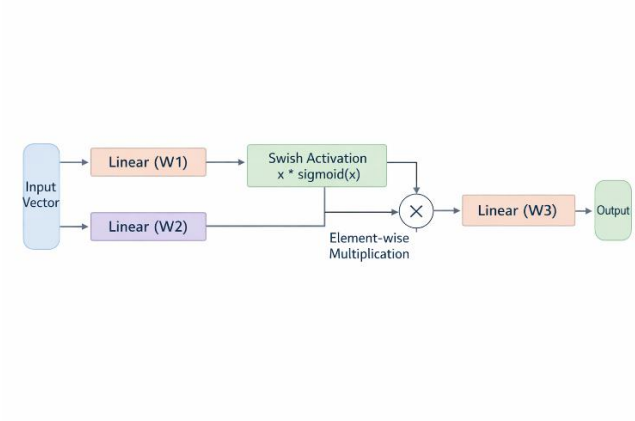
RESIDUAL CONNECTIONS

Residual connection in DOT are shortcut paths that add the input of each layer back to its output, ensuring information flows smoothly through the network. They help stabilize training, preserve earlier representations and allow stacking very deep layers without vanishing gradients.

FFN With SwiGLU

GLU (Gated Linear Unit) is a gating-based activation function where the input is transformed into two parallel linear projections, and one controls (gates) the other using a sigmoid function.

SwiGLU is a gated activation function that uses two linear projections of the same input, where one is passed through a Swish activation and then multiplied element-wise with the other to control information flow.



The input to SwiGLU is the RMS-normalized result of the residual connection between the original input and the masked multi-head attention output.

Mathematical Formulation

Step 1: Input

$$x \in \mathbb{R}^d$$

Step 2: Two Parallel Linear Transformations

$$z_1 = xW_1$$
$$z_2 = xW_2$$

Step 3: Apply Swish (SiLU) Activation

$$\mathrm{SiLU}(z_1) = z_1 \cdot \sigma(z_1)$$

Step 4: Element-wise Multiplication (Gating)

$$h = \mathrm{SiLU}(z_1) \odot z_2$$

Step 5: Final Linear Projection

$$y = hW_3$$

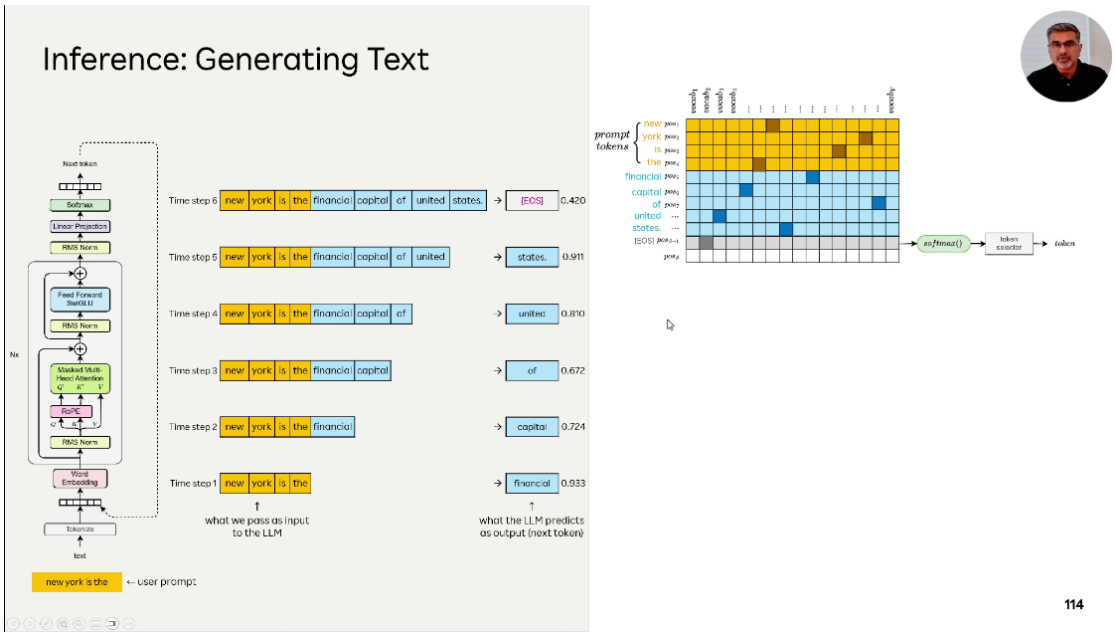
Compact Single Equation

$$\text{SwiGLU}(x) = (\text{SiLU}(xW_1) \odot (xW_2))W_3$$

LINEAR PROJECTION

The final linear projection layer in LLaMA is a learned linear transformation that maps each token's hidden representation from the model dimension to the vocabulary space, producing logits for next-token prediction.

HOW DOES LLM DECIDE WHICH TOKEN TO EMIT NEXT



Various Token Selection Strategies

Token selection strategies define how we pick the next token from the model's probability distribution during autoregressive generation.

Various methods are -

- Greedy Search
- Beam Search
- Top-K Sampling
- Top-P (Nucleus) Sampling

Greedy Search

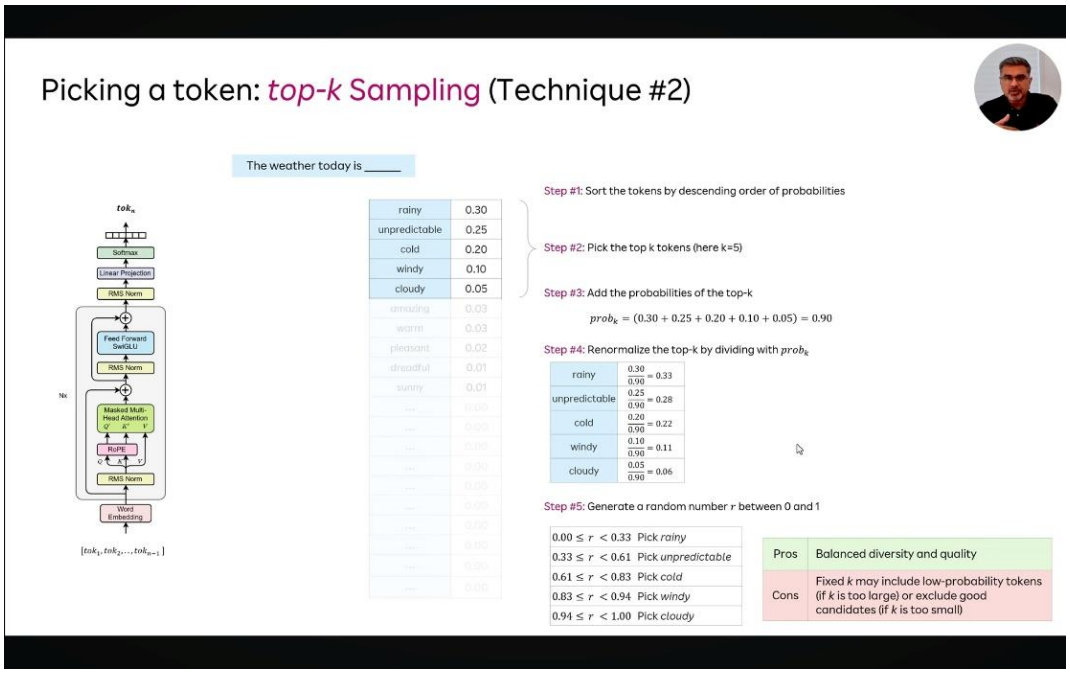
Greedy search is a decoding strategy where, at each step, the model selects the token with the highest probability (argmax) from the predicted distribution.

Working steps are –

1. Take input tokens and pass through the model
2. Get logits for the next token
3. Select token with highest value (**argmax**)
4. Append the selected token to the sequence
5. Repeat until EOS or max length is reached

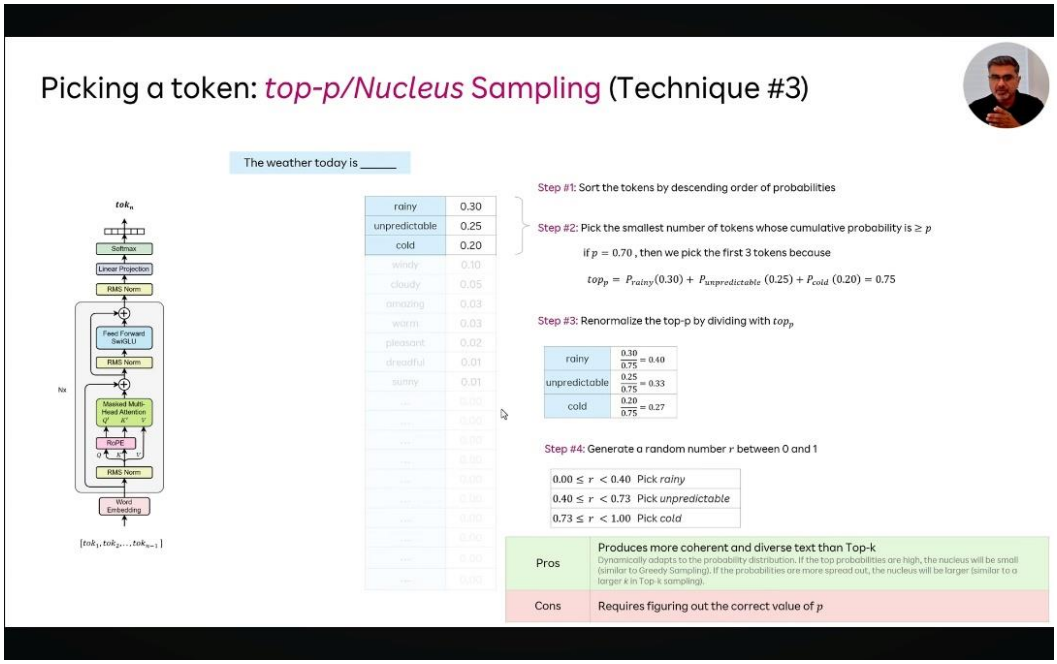
Top – K sampling

Top-K sampling is a decoding strategy where, at each step, the model restricts the candidate tokens to the top K highest-probability tokens and then samples the next token only from this reduced set.



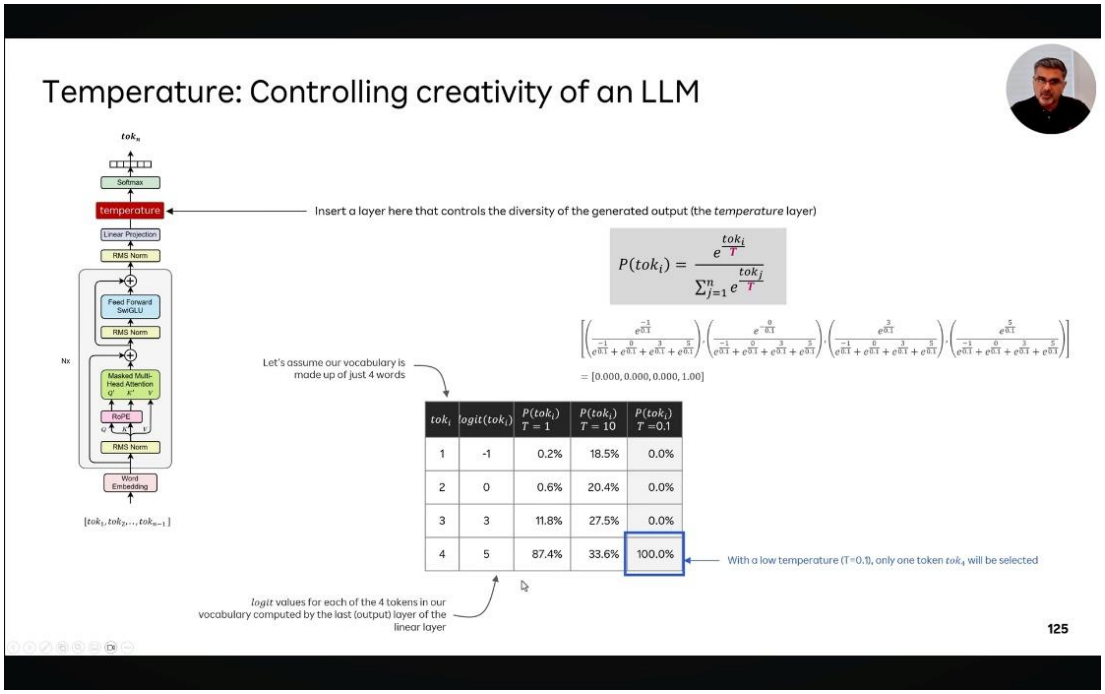
Top – P sampling

Top-P sampling (also called *nucleus sampling*) is a decoding strategy where, at each step, the model selects the smallest set of tokens whose cumulative probability ≥ p, and then samples the next token from this dynamically chosen set.

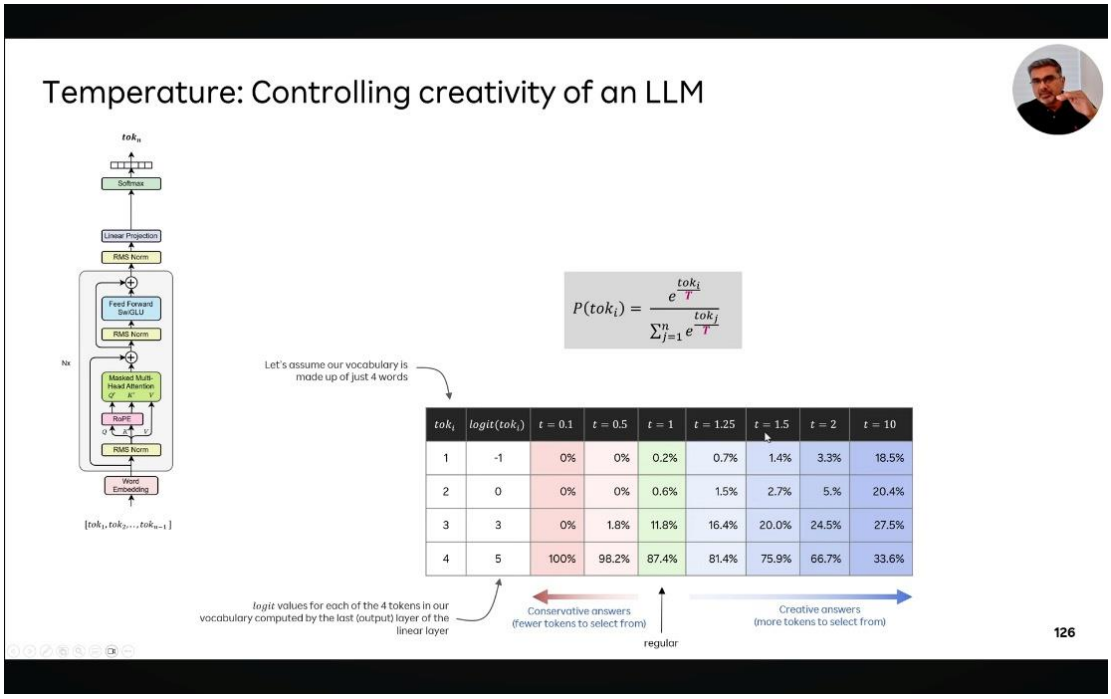


TEMPERATURE

Temperature controls the entropy of the output distribution by scaling logits before softmax, allowing us to adjust how deterministic or diverse the generated text is.



tok_i	$logit(tok_i)$	$P(tok_i)$ $T = 1$	$P(tok_i)$ $T = 10$	$P(tok_i)$ $T = 0.1$
1	-1	0.2%	18.5%	0.0%
2	0	0.6%	20.4%	0.0%
3	3	11.8%	27.5%	0.0%
4	5	87.4%	33.6%	100.0%



tok_i	$logit(tok_i)$	$t = 0.1$	$t = 0.5$	$t = 1$	$t = 1.25$	$t = 1.5$	$t = 2$	$t = 10$
1	-1	0%	0%	0.2%	0.7%	1.4%	3.3%	18.5%
2	0	0%	0%	0.6%	1.5%	2.7%	5.%	20.4%
3	3	0%	1.8%	11.8%	16.4%	20.0%	24.5%	27.5%
4	5	100%	98.2%	87.4%	81.4%	75.9%	66.7%	33.6%

HOW TO CHOOSE THE VALUE OF "T" ?

SO ask first - "Do I want accuracy or diversity?"

Use Case	Goal	Temperature
Code generation	Precision	0.1 – 0.4
QA / factual tasks	Correctness	0.2 – 0.5
Chatbots	Balanced	0.6 – 0.9
Creative writing	Diversity	0.9 – 1.3